

35 — Server-Sent Events (SSE) Streaming

What It Is

Server-Sent Events (SSE) is a one-way server-to-browser protocol where the server pushes events to the client as they happen. Unlike WebSockets, which are bidirectional (both sides can send messages at any time), SSE is simpler because only the server sends data — the client just listens.

Here is how it works at the HTTP level:

1. The browser opens a normal HTTP connection to the server (a `POST` or `GET` request).
2. Instead of sending one response and closing, the server keeps the connection open.
3. The server sends messages in a specific text format:

```
event: metadata
data: {"citations": [...], "detected_language": "hu"}

event: token
data: {"text": "The first few words"}

event: done
data: {"response_time_ms": 3241.7}
```

Each message has two lines: `event: <type>` names what kind of message it is, and `data: <json>` carries the payload. A blank line (two newlines `\n\n`) marks the end of one message. The browser's built-in `EventSource` API (or a manual `ReadableStream` reader) parses these lines automatically.

SSE uses plain HTTP — no special protocol upgrade like WebSockets requires. This means it works through proxies, load balancers, and CDNs without special configuration. The trade-off is that the client cannot send data back over the same connection (it would need a separate request for that), but for streaming LLM output that is exactly what we want.

Real-Life Analogy

Think of a live sports commentary radio broadcast:

- **You tune in** — you open the connection by turning on the radio (the browser sends the HTTP request).
- **The commentator speaks as things happen** — the server sends events one by one as the game progresses: "Goal scored!", "Yellow card!", "Half-time stats: possession 60-40".
- **You just listen** — the client receives events passively. You do not need to talk back to the commentator for the broadcast to continue.

If you wanted to request a replay or ask the commentator a question, you would need a different channel (a phone call, a text message) — that is a separate HTTP request in our analogy. The broadcast itself is one-way, which is exactly why it is so simple and reliable.

Why We Used It Here

RAG (Retrieval-Augmented Generation) answers can take 5 to 15 seconds to generate. The pipeline has two phases: (1) hybrid search retrieval (~200-500ms), then (2) LLM text generation (the slow part). Without streaming, the user stares at a blank screen for the entire duration — a terrible experience.

SSE lets us break the response into three stages that arrive progressively:

(a) Metadata arrives immediately after retrieval completes (~500ms)

As soon as the search results come back, we send a `metadata` event with citations, detected language, and retrieval mode. The UI can show "Sources found: [1] Category #42, [2] Company #17..." while the LLM is still thinking.

(b) Answer text streams word-by-word as the LLM generates it

Each `token` event carries a chunk of text (4 words at a time in our implementation). The UI appends each chunk to the message bubble in real time, giving the user the same "typing" effect they see in ChatGPT. The user can start reading the answer within 1-2 seconds of the LLM starting.

(c) A `done` event arrives with timing info

Once the full answer is generated, a final `done` event carries the total response time. The UI uses this to remove the "streaming" indicator and show performance badges.

This dramatically improves **perceived latency** — the user sees useful information within 500ms instead of waiting 10+ seconds for a complete response.

Code Walkthrough

1. The SSE Generator — `search_api/services/rag.py`

The streaming logic lives in the `rag_answer_stream()` generator function. A Python generator uses `yield` to produce values one at a time instead of returning everything at once — perfect for streaming.

The `_sse_event()` helper formats a single SSE message:

```
def sse_event(event: str, data: dict) -> str:
    return f"event: {event}\ndata: {json.dumps(data)}\n\n"
```

This produces the exact SSE wire format: event type on one line, JSON data on the next, and a blank line to terminate. Every message the server sends goes through this helper to guarantee consistent formatting.

The `STREAM_CHUNK_SIZE` constant controls how many words are sent per token event:

```
STREAM_CHUNK_SIZE = 4
```

Sending one word at a time would create too many tiny HTTP chunks (overhead per chunk). Sending 100 words at a time would defeat the purpose of streaming. Four words is a practical balance — the text appears to flow smoothly without excessive network overhead.

The generator function itself has three phases:

```
def rag_answer_stream(
    query: str,
    table: str = "category_vectors",
    limit: int = 5,
    lang_override: str | None = None,
) -> Generator[str, None, None]:
    start = time.perf_counter()

    # Phase 1: Retrieve documents (same as non-streaming path)
    raw = search_table(table=table, query=query, limit=limit,
                      lang_override=lang_override, rerank=True)

    results = raw["results"]
    context_text, citations = _build_context(results, table)
```

```

# Phase 2: Yield metadata immediately – client sees citations now
yield sse event("metadata", {
    "citations": citations,
    "detected language": raw["detected language"],
    "retrieval mode": raw["retrieval_mode"],
    "results_used": len(citations),
})

# Phase 3: Generate LLM answer, then chunk it into token events
llm = get_llm_provider()
answer = generate_with_retry(llm, user_prompt, system_prompt=RAG_S

words = answer.split(" ")
for i in range(0, len(words), STREAM_CHUNK_SIZE):
    chunk = " ".join(words[i:i + STREAM_CHUNK_SIZE])
    if i > 0:
        chunk = " " + chunk # preserve space between chunks
    yield _sse_event("token", {"text": chunk})

# Phase 4: Signal completion with timing
elapsed = (time.perf_counter() - start) * 1000
yield _sse_event("done", {"response_time_ms": round(elapsed, 1)})

```

Key detail: `generate_with_retry` currently returns the full answer at once, then the code splits it into word chunks. In a future iteration, if the LLM provider supports true token-by-token streaming, the chunking loop would be replaced with an async iterator over the provider's stream — but the SSE event format stays exactly the same.

2. The FastAPI Route — `search_api/routers/rag.py`

The route handler checks `req.stream` to decide which path to take:

```

@router.post("/ask")
def ask(req: RAGRequest):
    if req.stream:
        logger.info(f"EVIDENCE RAG_STREAM: query={req.query!r} stream=
        return StreamingResponse(
            rag_answer_stream(
                query=req.query,
                table=req.table,
                limit=req.limit,
                lang_override=req.language,
            ),
            media_type="text/event-stream",
            headers={"Cache-Control": "no-cache", "X-Accel-Buffering":

```

```
)  
# ... non-streaming path returns a normal JSON response
```

Three things to notice:

- **StreamingResponse** is FastAPI's wrapper that takes a generator and sends each yielded string as an HTTP chunk. It keeps the connection open until the generator is exhausted.
- **media_type="text/event-stream"** tells the browser "this is an SSE stream, not a JSON blob". The browser (or our manual reader) knows to parse `event: / data:` lines.
- **Cache-Control: no-cache** and **X-Accel-Buffering: no** are critical headers. Without them, reverse proxies (like nginx) or browser caches might buffer the entire response before delivering it — which would defeat streaming entirely. **X-Accel-Buffering: no** specifically tells nginx to pass chunks through immediately.

3. The React Consumer — `admin-ui/src/pages/RAGChat.jsx`

The `handleSend` function in the `RAGChat` component consumes the SSE stream. It uses the low-level `fetch()` + `ReadableStream` API instead of the browser's `EventSource` because `EventSource` only supports GET requests, and our endpoint is a POST.

Opening the connection:

```
const resp = await fetch("/ask", {  
  method: "POST",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify({ query: q, table, limit, stream: true }),  
});
```

Reading chunks from the stream:

```
const reader = resp.body.getReader();  
const decoder = new TextDecoder();  
let buffer = "";  
let meta = {};  
let fullText = "";  
  
while (true) {  
  const { done, value } = await reader.read();  
  if (done) break;  
  buffer += decoder.decode(value, { stream: true });
```

```
const lines = buffer.split("\n");
buffer = lines.pop() || ""; // keep incomplete last line for next i
```

The `reader.read()` loop pulls raw bytes from the HTTP stream as they arrive. `TextDecoder` converts bytes to a string. Because a network chunk might arrive mid-line (for example, `"event: tok"` in one chunk and `"en\ndata: ..."` in the next), we keep a `buffer` and only process complete lines. The last element from `split("\n")` might be an incomplete line, so we put it back in the buffer.

Parsing SSE events:

```
let eventType = "";
for (const line of lines) {
  if (line.startsWith("event: ")) {
    eventType = line.slice(7).trim();
  } else if (line.startsWith("data: ") && eventType) {
    const payload = JSON.parse(line.slice(6));
    if (eventType === "metadata") {
      meta = payload;
    } else if (eventType === "token") {
      fullText += payload.text;
      setMessages((prev) => {
        const updated = [...prev];
        updated[assistantIdx] = {
          role: "assistant",
          text: fullText,
          citations: meta.citations,
          streaming: true,
        };
        return updated;
      });
    } else if (eventType === "done") {
      // Final update: add timing metadata, clear streaming flag
    }
    eventType = "";
  }
}
```

The parser is a simple state machine: when it sees an `event:` line, it saves the type. When it sees the `data:` line, it processes the payload based on the saved type, then resets. For each `token` event, it appends the text to `fullText` and calls `setMessages` to update React state — this triggers a re-render that shows the new text immediately in the chat bubble.

The `streaming: true` flag on the message object lets the UI show a subtle animation or indicator while text is still arriving. When the `done` event arrives, it flips to `streaming: false` and adds timing badges.

4. The Vite Proxy — `admin-ui/vite.config.js`

During development, the React app runs on port 5173 (Vite dev server) and the FastAPI backend runs on port 8000. The browser sends requests to port 5173, so we need a proxy to forward API calls:

```
export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      "/ask": "http://localhost:8000",
      // ... other proxied routes
    },
  },
});
```

The `/ask` entry is especially important for SSE because:

1. Without the proxy, the browser would need to make a cross-origin request to port 8000, triggering CORS preflight checks.
2. Some proxy configurations buffer responses before forwarding them. Vite's built-in proxy uses `http-proxy`, which passes chunks through by default — so SSE works without extra configuration.
3. In production (Docker), the reverse proxy (or direct container networking) replaces this, but the same "no buffering" requirement applies.

How to Explain This in an Interview / to a Teammate

"Our RAG endpoint can take 5-15 seconds because it runs a hybrid search and then waits for the LLM to generate an answer. Instead of making the user wait for the whole thing, we use Server-Sent Events to stream the response in stages. As soon as the search finishes, we push the citations and metadata to the browser. Then, as the LLM produces text, we send it in small chunks that the UI renders word-by-word — like watching someone type in real time. SSE is just plain HTTP with a special text format (`event: / data: lines`), so it works through any proxy or load balancer without special setup. On the frontend, we read the stream with the Fetch API's `ReadableStream` reader and update React state for each chunk, which gives us a ChatGPT-style streaming experience with minimal code."